

CS1632: Unit Testing, part 2

Wonsun Ahn

Mocks Allow True Unit Testing

By removing object dependencies from unit testing

Unit Testing Control.getInput() with Dependencies

System

```
class Game {  
    public static void main() {  
        control.getInput();  
        display.show();  
    }  
}
```

Subsystems

Unit Test

```
class Control {  
    public String getInput() {  
        mouse.getInput();  
        keyboard.getInput();  
    }  
}
```

```
class Display {  
    public void show() {  
        scenery.show();  
    }  
}
```

Modules

```
class Mouse {  
    public String getInput() {  
        ...  
    }  
}
```

```
class Keyboard {  
    public String getInput() {  
        ...  
    }  
}
```

```
class Scenery {  
    public void show() {  
        ...  
    }  
}
```

First, let's get rid of irrelevant classes

System

```
class Control {  
    public static void main()  
    {  
        control.getInput();  
        display.show();  
    }  
}
```

Subsystems

Unit Test

```
class Control {  
    public String getInput() {  
        mouse.getInput();  
        keyboard.getInput();  
    }  
}
```

```
class Display {  
    public void show() {  
        scenery.show();  
    }  
}
```

Modules

```
class Mouse {  
    public String getInput() {  
        ...  
    }  
}
```

```
class Keyboard {  
    public String getInput() {  
        ...  
    }  
}
```

```
class Scenery {  
    public void show() {  
        ...  
    }  
}
```

What should we do with dependencies?

System

Subsystems

Unit Test

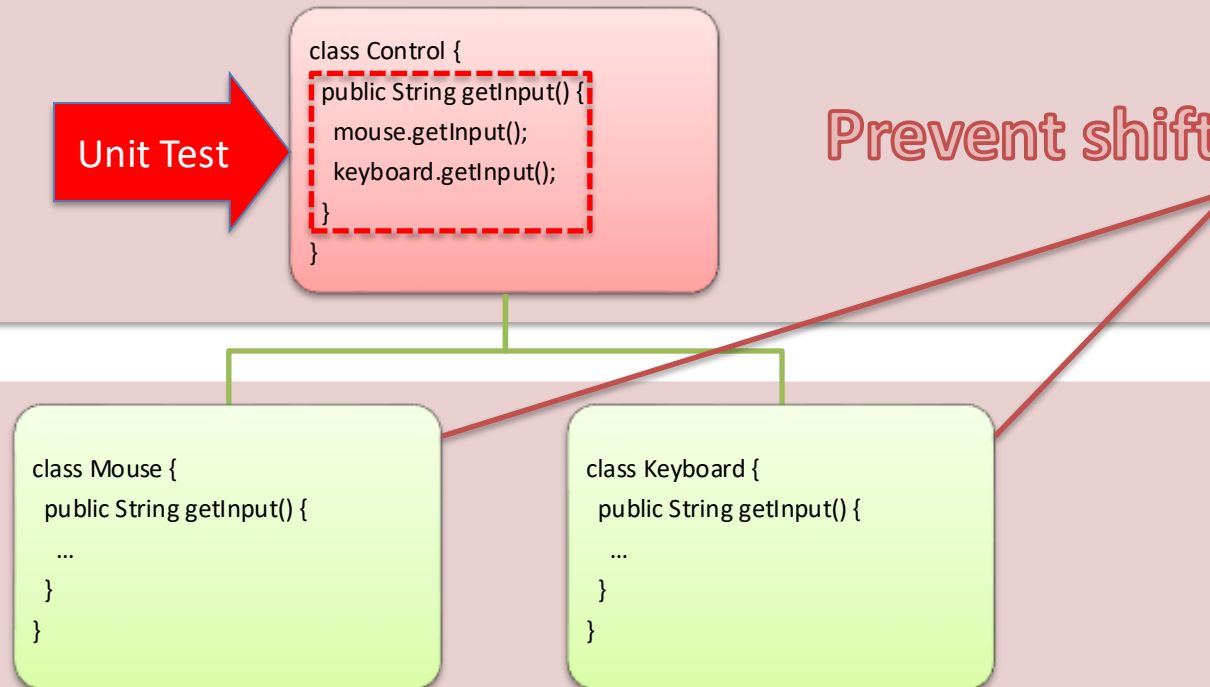
```
class Control {  
  public String getInput() {  
    mouse.getInput();  
    keyboard.getInput();  
  }  
}
```

Prevent shift-left testing!

Modules

```
class Mouse {  
  public String getInput() {  
    ...  
  }  
}
```

```
class Keyboard {  
  public String getInput() {  
    ...  
  }  
}
```



Answer: Replace dependencies with fake objects!

System

Subsystems

Unit Test

```
class Control {  
  public String getInput() {  
    mouse.getInput();  
    keyboard.getInput();  
  }  
}
```

Allows shift-left testing,
since no code is executed.

Modules

Fake Mouse

```
String getInput() {  
  // No code  
}
```

Fake Keyboard

```
String getInput() {  
  // No code  
}
```

Test Doubles fake the real object

- Goal: To **not execute code** in external classes as part of the unit test.
 - Means method can be **shift-left** tested.
 - Means if a defect is found, it is **localized**.
- Just like body doubles, **test doubles** pretend to be the real thing but aren't.
 - Mockito **mocks** (mock-ups of real objects), is a type of test double.
 - Mocks appear to be real objects, but without executing any code. How???
 - Hint: mock only needs to emulate behavior for the **given test scenario**.

Running Example: Rent-A-Cat System

```
class RentACat {
    HashMap<Integer, Cat> cats = ...;

    public void addCat(int id, Cat cat) {
        cats.put(id, cat);
    }
    public void rentCat(int id, int days) {
        cats.get(id).pay(days * 100);
    }
    public String listCats() {
        String ret = "";
        for (Cat cat : cats.values()) {
            ret += cat.toString() + "\n";
        }
        return ret;
    }
}
```

```
class Cat {
    String name;
    int netWorth = 0;

    public Cat(String name) {
        this.name = name;
    }
    public void pay(int payment) {
        netWorth += payment;
    }
    public String toString() {
        return name + " " + netWorth;
    }
}
```

RentACat depends on Cat

```
class RentACat {  
    HashMap<Integer, Cat> cats = ...;  
  
    public void addCat(int id, Cat cat) {  
        cats.put(id, cat);  
    }  
    public void rentCat(int id, int days) {  
        cats.get(id).pay(days * 100);  
    }  
    public String listCats() {  
        String ret = "";  
        for (Cat cat : cats.values()) {  
            ret += cat.toString() + "\n";  
        }  
        return ret;  
    }  
}
```

```
class Cat {  
    String name;  
    int netWorth = 0;  
  
    public Cat(String name) {  
        this.name = name;  
    }  
    public void pay(int payment) {  
        netWorth += payment;  
    }  
    public String toString() {  
        return name + " " + netWorth;  
    }  
}
```

How can we test RentACat w/o Cat code?

```
class RentACat {  
    HashMap<Integer, Cat> cats = ...;  
  
    public void addCat(int id, Cat cat) {  
        cats.put(id, cat);  
    }  
    public void rentCat(int id, int days) {  
        cats.get(id).pay(days * 100);  
    }  
    public String listCats() {  
        String ret = "";  
        for (Cat cat : cats.values()) {  
            ret += cat.toString() + "\n";  
        }  
        return ret;  
    }  
}
```

"Mock" Cat

"Fake" void pay(int payment)

"Fake" String toString()

Mockito Framework

A popular Java framework for creating mock objects

Mock: A Fake Object with No Code

```
// Creates a mock cat of type Cat  
Cat cat = Mockito.mock(Cat.class);
```

Mockito: a framework for creating test doubles

- **Mockito**: a framework for creating mocks
 - Good for emulating test doubles that exhibit simple behaviors
 - Uses Java Reflection + Bytecode Rewriting to override method behavior
 - Yes, method bytecode of mocks is literally rewritten during the test!
- In Mockito terminology:
 - Test double → **Mock**, Act of creating a mock → **Mocking**

A Mock Object does not execute your code!

```
Cat cat = new Cat("Tabby");
```

```
class Cat {  
    String name;  
    int netWorth = 0;  
  
    public Cat(String name) {  
        this.name = name;  
    }  
    public void pay(int payment) {  
        netWorth += payment;  
    }  
    public String toString() {  
        return name + " " + netWorth;  
    }  
}
```

```
Cat cat = Mockito.mock(Cat.class);
```

```
// No member variables  
// No constructor  
  
// Default code for pay  
void pay(int payment) {}  
// Default code for toString  
String toString() {  
    return "Mock for Cat, ...";  
}
```

Stub: A Fake Method with No Code

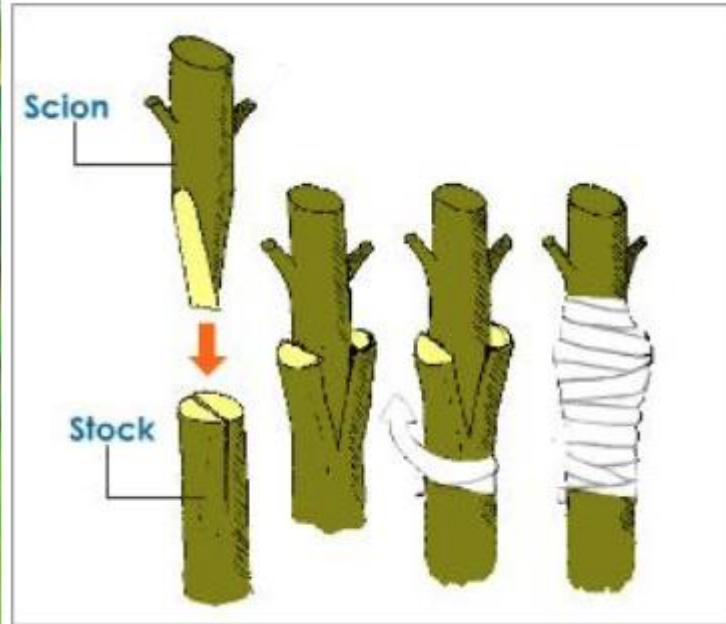
```
// Rewrites toString() stub to return "Tabby 0"  
Mockito.when(cat.toString()).thenReturn("Tabby 0");
```

Stubbing Getter Methods Emulates State

- OOP 101: objects have *data encapsulation*
 - *Data encapsulation*: private members are not visible from outside
 - Only way to query private state is through *getter* methods
 - *Getter*: a method that returns the value of a private member variable
- What if a **precondition** specifies a state for a mock object?
 - E.g. Cat has a name of “Tabby” and a net worth of 300 dollars.
 - Answer: rewrite *getter* methods to return specified state!

Stubbing sets up preconditions.

- In Mockito terminology:
 - Fake method → **Stub**, Act of changing method return value → **Stubbing**



Courtesy: Bainbridge Island Fruit Club

- Grafts apple tree limb to the stub of another tree.
- For all purposes, tree acts like an apple tree!
 - If precondition says red apples, stub red apples
 - If precondition says green apples, stub green apples

Creating a cat named "Tabby"

```
Cat cat = new Cat("Tabby");
```

```
class Cat {  
    String name;  
    int netWorth = 0;  
  
    public Cat(String name) {  
        this.name = name;  
    }  
    public void pay(int payment) {  
        netWorth += payment;  
    }  
    public String toString() {  
        return name + " " + netWorth;  
    }  
}
```

```
Cat cat = Mockito.mock(Cat.class);  
Mockito.when(cat.toString()).thenReturn("Tabby 0");
```

```
void pay(int payment) {}  
  
// Now returns "Tabby 0"!  
String toString() {  
    return "Mock for Cat, ...";  
    return "Tabby 0";  
}
```

Creating a cat named “Tabby” with net worth 5

```
Cat cat = new Cat("Tabby");  
cat.pay(5);
```

```
Cat cat = Mockito.mock(Cat.class);  
Mockito.when(cat.toString()).thenReturn("Tabby 5");
```

```
class Cat {  
    String name;  
    int netWorth = 0;  
  
    public Cat(String name) {  
        this.name = name;  
    }  
    public void pay(int payment) {  
        netWorth += payment;  
    }  
    public String toString() {  
        return name + " " + netWorth;  
    }  
}
```

```
void pay(int payment) {}  
  
// Now returns "Tabby 5!"  
String toString() {  
    return "Mock for Cat, ...";  
    return "Tabby 5";  
}
```

Integration Testing listCats()

```
class IntegrationTest {  
    @Test  
    public void testListCats() {  
        // Preconditions: System has a cat named "Tabby", net worth 300, ID 1.  
        RentACat rentACat = new RentACat();  
        Cat cat = new Cat("Tabby");  
        rentACat.addCat(1, cat);  
        rentACat.rentCat(1, 3);  
        // Execution Steps: List all cats in the system.  
        String str = rentACat.listCats();  
        // Postconditions: "Tabby" is listed with net worth 300  
        assertEquals("Tabby 300\n", str);  
    }  
}
```

Unit Testing listCats()

```
class RentACatUnitTest {
    @Test
    public void testListCats() {
        // Preconditions: System has a cat named "Tabby", net worth 300, ID 1.
        RentACat rentACat = new RentACat();
        Cat cat = Mockito.mock(Cat.class);
        Mockito.when(cat.toString()).thenReturn("Tabby 300");
        rentACat.addCat(1, cat);
        // Execution Steps: List all cats in the system.
        String str = rentACat.listCats();
        // Postconditions: "Tabby" is listed with net worth 300
        assertEquals("Tabby 300\n", str);
    }
}
```

Behavior Verification:

Allows postcondition checks on Mocks

```
// Verifies pay(300) has been called on cat  
Mockito.verify(cat).pay(300);
```

Integration Test for rentCat()

```
class IntegrationTest {
    @Test
    public void testRentCat() {
        // Preconditions: System has cat named "Tabby", net worth 0, ID 1.
        RentACat rentACat = new RentACat();
        Cat cat = new Cat("Tabby");
        rentACat.addCat(1, cat);
        // Execution Steps: Rent out "Tabby" for 3 days (100 USD / day).
        rentACat.rentCat(1, 3);
        // Postconditions: "Tabby" has net worth 300
        assertEquals("Tabby 300", cat.toString());
    }
}
```

Unit Test for rentCat() that FAILS

```
class RentACatUnitTest {
    @Test
    public void testRentCat() {
        // Preconditions: System has cat named "Tabby", net worth 0, ID 1.
        RentACat rentACat = new RentACat();
        Cat cat = Mockito.mock(Cat.class);
        Mockito.when(cat.toString()).thenReturn("Tabby 0");
        rentACat.addCat(1, cat);
        // Execution Steps: Rent out "Tabby" for 3 days (100 USD / day).
        rentACat.rentCat(1, 3);
        // Postconditions: "Tabby" has net worth 300 => FAILS! Why?
        assertEquals("Tabby 300", cat.toString());
    }
}
```

Mock state cannot (and should not) be checked

1. **Cannot** be done.

- Mock cat has no state so there is nothing to check.
- What if we emulated the state to check through stubbing?

```
Mockito.when(cat.toString()).thenReturn("Tabby 300");  
assertEquals("Tabby 300", cat.toString());
```

This is called a *tautological test*, because it always passes regardless of defects.

2. **Should not** be done.

- You are checking something about Cat, which is beyond the scope of testing.

Behavior of Test Target on Mock *can* be checked

1. **Can** be done.

- Mockito framework keeps track of all calls to mock objects.
- Check rentACat (test target) calls pay on cat (mock) with correct payment:

```
Mockito.verify(cat).pay(payment);
```

```
Mockito.verify(cat, Mockito.times(1)).pay(payment);
```

2. **Should** be done.

- You are verifying something about rentACat, not cat.

Unit Test for rentCat() that **PASSES**

```
class RentACatUnitTest {
    @Test
    public void testRentCat() {
        // Preconditions: System has cat named "Tabby", net worth 0, ID 1.
        RentACat rentACat = new RentACat();
        Cat cat = Mockito.mock(Cat.class);
        Mockito.when(cat.toString()).thenReturn("Tabby 0");
        rentACat.addCat(1, cat);
        // Execution Steps: Rent out "Tabby" for 3 days (100 USD / day).
        rentACat.rentCat(1, 3);
        // Postconditions: "Tabby" has net worth 300
        // Postconditions: "Tabby" is given payment of 300
        Mockito.verify(cat).pay(300);
    }
}
```

Pitfall 1: Using Mockito.verify() on Test Target

1. **Cannot** be done.*

- The test target is always a real object (no point in testing a mock object, right?)
- Mockito.verify only works on mock objects.

2. **Should not** be done.

- Is it necessary to check that tested method call a “helper” method inside?
No! The helper modifies observable state that is checked using **state verification**.

* Mockito.spy allows even real objects to be monitored using Mockito.verify()

- Per Mockito manual: “spies should be used **carefully and occasionally**”.
- Outside this scope of this class (you will have no reason to use it in this class).

Is this a good use of Mockito.verify()?

```
class RentACatUnitTest {
    @Test
    public void testListCats() {
        // Preconditions: System has cat named "Tabby", net worth 300, ID 1.
        RentACat rentACat = new RentACat();    Cat cat = Mockito.mock(Cat.class);
        Mockito.when(cat.toString()).thenReturn("Tabby 300");
        rentACat.addCat(1, cat);
        // Execution Steps: List all cats in the system.
        String str = rentACat.listCats();
        // Postconditions: The list consists of "Tabby" with net worth 300.
        assertEquals("Tabby 300\n", str); // This is the observable behavior!
        Mockito.verify(cat).toString(); // Is this a good use of verify()?
    }
}
```

Only “setter” methods need verification

- In summary, the “behavior” in behavior verification is:
behavior of the test target that impacts external (mock) objects
 - Behavior that impacts test target state → verified using **state verification**
(Using assertEquals, assertTrue, assertNotNull, ...)
- Only “setter” methods have observable impact on external objects
 - E.g. `pay(int payment)` method call has an observable impact on cat.
- “Getters” have no observable impact so should not be verified
 - E.g. `toString()` method call has no observable impact on cat
→ since it is not observable behavior, should not be a postcondition

Pitfall 2: Using Verify on a Getter Method

```
class RentACatUnitTest {
    @Test
    public void testListCats() {
        // Preconditions: System has cat named "Tabby", net worth 300, ID 1.
        RentACat rentACat = new RentACat();    Cat cat = Mockito.mock(Cat.class);
        Mockito.when(cat.toString()).thenReturn("Tabby 300");
        rentACat.addCat(1, cat);
        // Execution Steps: List all cats in the system.
        String str = rentACat.listCats();
        // Postconditions: The list consists of "Tabby" with net worth 300.
        assertEquals("Tabby 300\n", str); // This is the observable behavior!
        Mockito.verify(cat).toString(); // Not observable behavior of listCats().
    }
}
```

Getter methods are not targets of verification

```
class RentACat {  
    HashMap<Integer, Cat> cats = ...;  
  
    public void addCat(int id, Cat cat) {  
        cats.put(id, cat);  
    }  
    public void rentCat(int id, int days) {  
        cats.get(id).pay(days * 100);  
    }  
    public String listCats() {  
        String ret = "";  
        for (Cat cat : cats.values()) {  
            ret += cat.toString() + "\n";  
        }  
        return ret;  
    }  
}
```

Verify?

[Mock Cat]

```
void pay(int payment) {}  
String toString() {}  
// Let's say Cat also had  
// these interfaces.  
String getName() {...}  
int getNetWorth() {...}
```

Postconditions should check observable behavior of `listCats()`.
Calling `toString()` has no observable outcome.

Verifying getter methods only causes spurious failures

```
class RentACat {  
    ...  
    // New version that still works  
    public String listCats() {  
        String ret = "";  
        for (Cat cat : cats.values()) {  
            ret += cat.getName() + " " +  
                cat.getNetWorth() + "\n";  
        }  
        return ret;  
    }  
}
```

[Mock Cat]

```
void pay(int payment) {...}  
String toString() {...}  
// Let's say Cat also had  
// these interfaces.  
String getName() {...}  
int getNetWorth() {...}
```

Verifying toString() fails even when listCats() still works.

The Hidden Costs of Mocking

- Doesn't verifying setter methods have the same problem?
 - That changes in how objects interact with each other can break tests?
 - Yes! And here lies the central problem behind mocking:
 - It checks interactions between units, not behavior of the integrated system
1. When interactions change while system behavior remains the same
 - **Can cause spurious test failures** → brittle tests need constant maintenance
 2. When these checks do not guarantee correct system behavior
 - **Defects can go undetected**, when mocked units don't behave like real units
 - Exactly why integration testing is needed!

Fake Class: Test Double for Dynamic Behavior

Mockito stubbing is not ideal for dynamic behavior

We can't stub `cat.toString()` like we used to

```
class RentACat {
    HashMap<Integer, Cat> cats = ...;

    public void addCat(int id, Cat cat) {
        cats.put(id, cat);
    }
    // cat.toString() needs 2 return values.
    // Not ideal for Mockito.when stubbing!*
    public String rentCat(int id, int days) {
        Cat cat = cats.get(id);
        String ret = cat.toString() + "\n";
        cat.pay(days * 100);
        ret += cat.toString() + "\n";
        return ret;
    }
}
```

```
class Cat {
    String name;
    int netWorth = 0;

    public Cat(String name) {
        this.name = name;
    }
    public void pay(int payment) {
        netWorth += payment;
    }
    public String toString() {
        return name + " " + netWorth;
    }
}
```

* Mockito does support dynamic behavior using APIs like `doAnswer()`, but the resulting code is far from clean - Fake Classes are often cleaner.

Create a Fake Class that emulates minimal behavior for given test scenario.

```
class RentACatUnitTest {
    @Test
    public void testRentCat3Days() {
        RentACat rentACat = new RentACat();

        Cat cat = new FakeCat3Days("Tabby");
        rentACat.addCat(1, cat);

        String str = rentACat.rentCat(1, 3);

        assertEquals("Tabby 0\nTabby 300\n", str);
    }
}
```

```
class FakeCat3Days extends Cat {
    String ret = "Tabby 0";

    public FakeCat3Days(String name) {
        super(name);
    }

    public void pay(int payment) {
        ret = "Tabby 300";
    }

    public String toString() {
        return ret;
    }
}
```

Another Fake Class for Another Test Scenario

```
class RentACatUnitTest {
    @Test
    public void testRentCat5Days() {
        RentACat rentACat = new RentACat();

        Cat cat = new FakeCat5Days("Tabby");
        rentACat.addCat(1, cat);

        String str = rentACat.rentCat(1, 5);

        assertEquals("Tabby 0\nTabby 500\n", str);
    }
}
```

```
class FakeCat5Days extends Cat {
    String ret = "Tabby 0";

    public FakeCat5Days(String name) {
        super(name);
    }

    public void pay(int payment) {
        ret = "Tabby 500";
    }

    public String toString() {
        return ret;
    }
}
```

How to Create a Fake Class

- Inherit from class you want to fake
- Override methods to remove as much code as possible
- Insert minimum amount of code to emulate correct behavior

Other Uses of Mockito

Unit Testing is not the only use of Mockito

Mocking has Uses Other than Unit Testing

- Faster testing: mock parts of system that slow down testing
 - Parts accessing databases, file systems, networks
- Robustness testing: for emulating hardware device failures
 - Hard to induce failures in real devices such as hard disks
 - Emulate failure in mock device to test how the system responds
- Repeatable testing: for making nondeterministic software deterministic
 - Hard to test programs that rely on random number generators
 - Decide exactly what numbers get generated using mock generators

Now Please Read Textbook Chapter 14

- Mockito User Manual:

<https://javadoc.io/static/org.mockito/mockito-core/3.2.4/org/mockito/Mockito.html>

- Try `sample_code/junit_example` to get some practice

- Further reading: “Mocks Aren't Stubs”:

<https://martinfowler.com/articles/mocksArentStubs.html>

- Compares the Mockist and Classicist schools of unit testing
- Classicists do unit testing with no mocking at all